IIMT3642: Managing and Mining Big Data - Recommender System

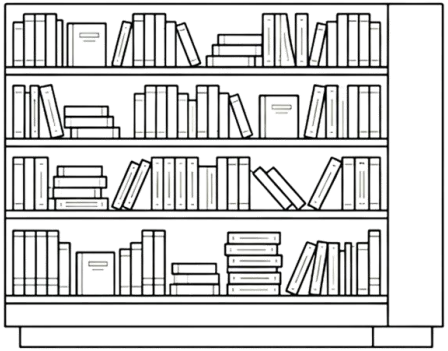
Zhepeng (Lionel) Li, PhD
Associate Professor
Innovation and Information Management

Introduction to Recommender Systems (RS)

How do you decide what to watch on Netflix or what to buy on Amazon?

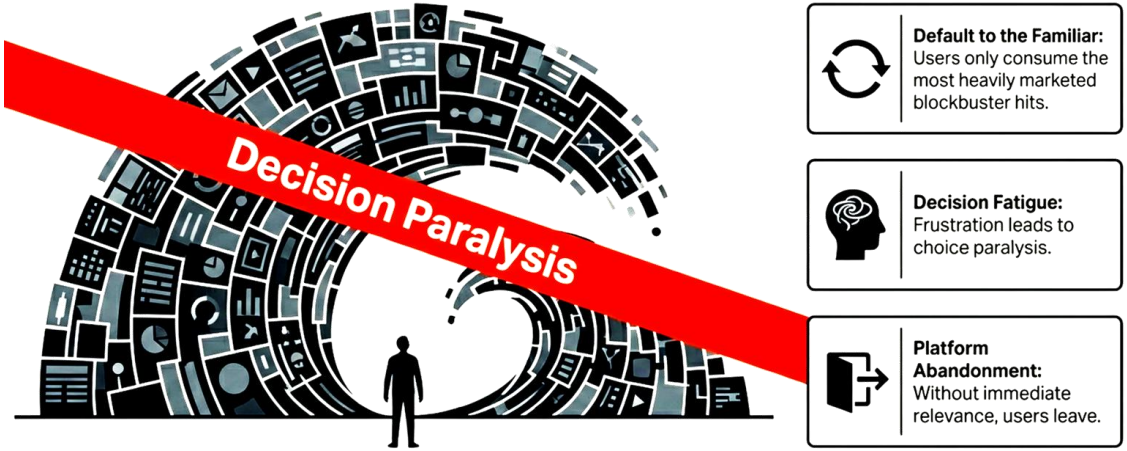
- The Era of Infinite Choice
  - Digital platform has removed the physical constraints of traditional retail
    - For example, shelf space is no longer a limiting factor

Physical Scarcity



Digital Abundance

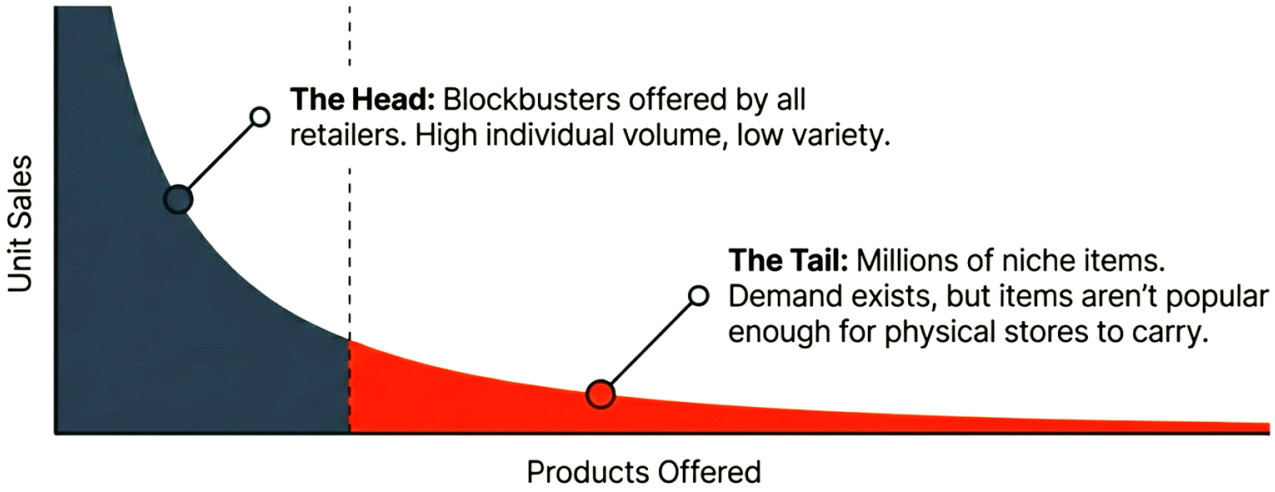
- Information Overload – users are overwhelmed by the abundance of data
  - While adequate information abundance is appreciated, endless choices actually create a severe cognitive burden for the user
  - The sheer volume of choices exceeds human capacity to process them, leading to frustration and decision paralysis



- Information Filtering – the process of selecting and presenting only the most relevant information to a user from a large pool of available data
  - Reducing noise
  - Prioritizing relevance
  - Personalizing the information space
- Recommender System (RS) is a type of information filtering system that focuses on personalizing item or content based on user preferences
  - Estimate users' preference or rating for items they have not yet seen
- Recommender system helps
  - Alleviate the information overload
  - Improve for business performance

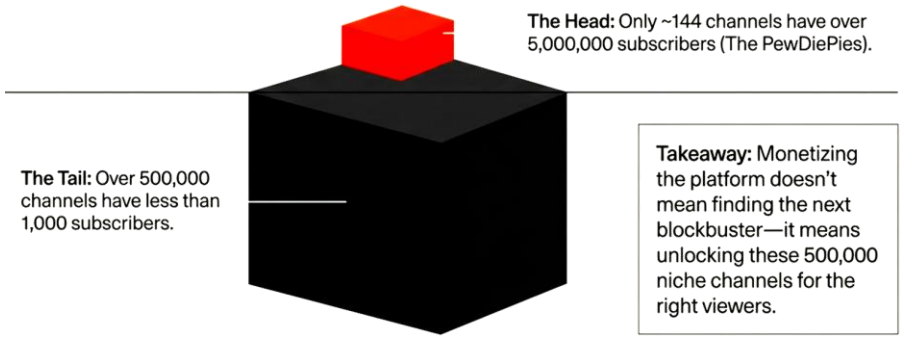
Business Value

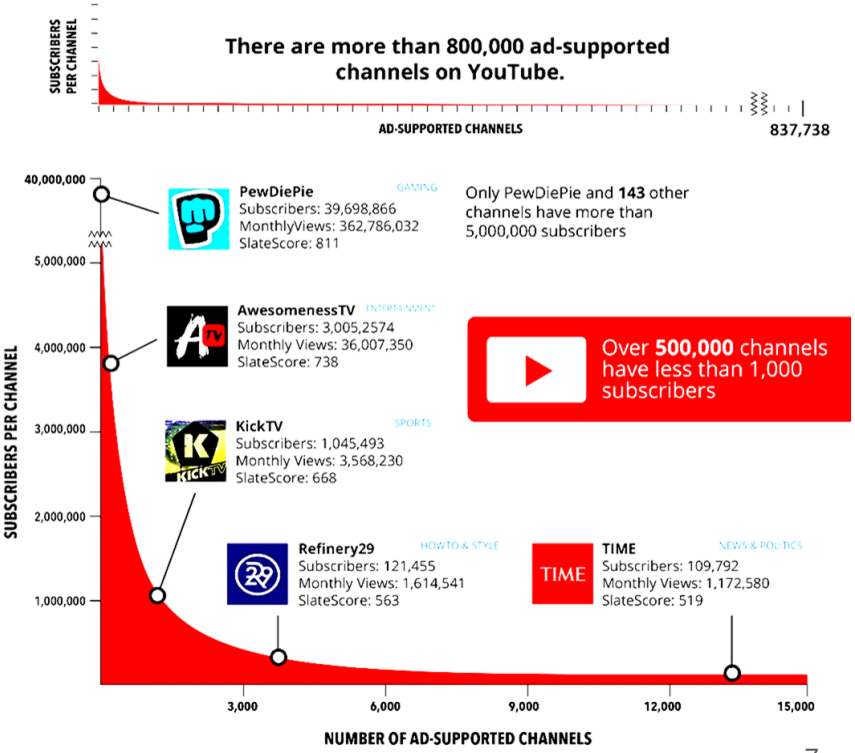
- The Long Tail – Secret to digital profitability



Takeaway: The aggregate demand for millions of niche items often equals or exceeds the total demand for the blockbuster hits.

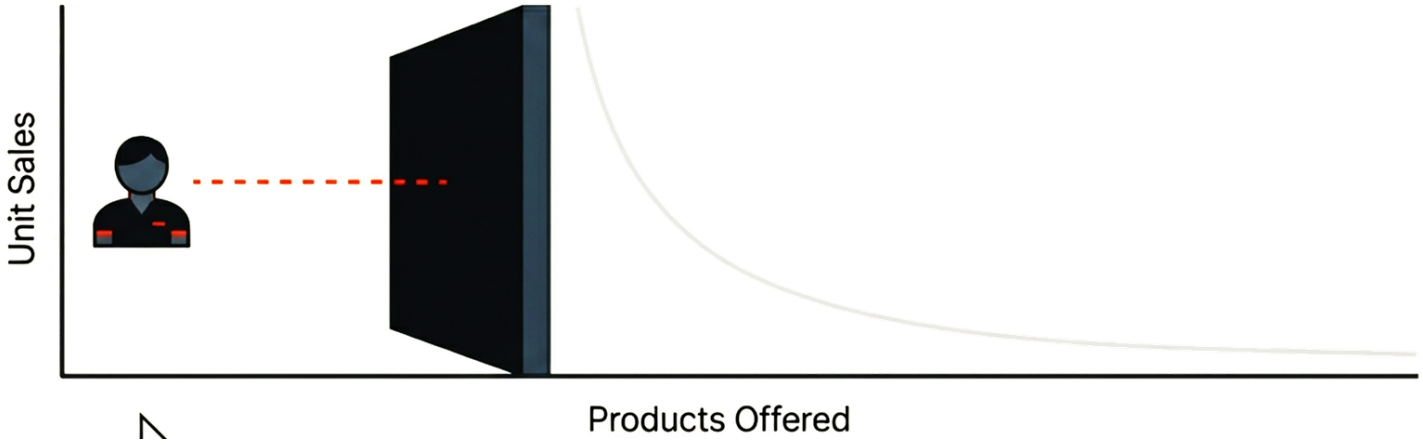
Example: the Long Tail at YouTube





The Discovery Gap

- How does a platform seamlessly match a user with the exact niche item they did not even know they wanted, from an inventory of millions?
  - Organic browsing (without RS) with top charts, trending lists, search results, homepage highlights are all biased towards popular items



Recommendation Problem

RS can be formalized as a function $f(user, item) = score$ where

- Given:
  - User model (e.g. ratings, preferences, demographics, context)
  - Items (with or without description of item characteristics)
- It estimates:
  - Relevance score of each item to a specified user. Then we can rank items accordingly for a user.

Recommendation Methods

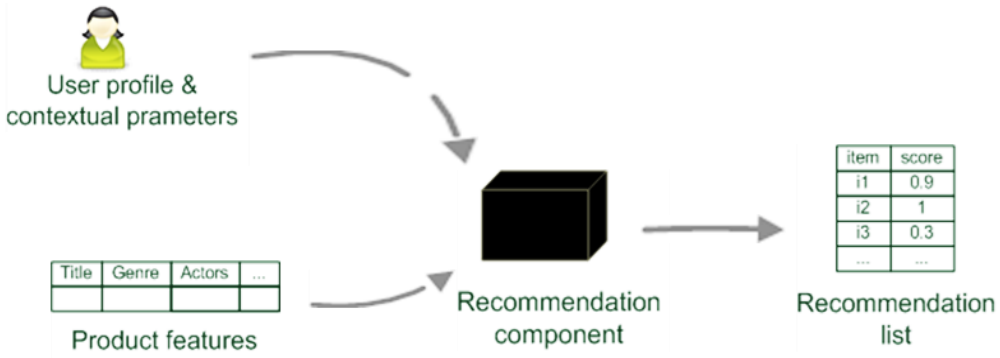
- Non-personalized methods
  - Top popularity
  - Top rated

- Personalized methods
  - Content-based recommendation
  - Collaborative filtering
  - Latent factor models

Recommendation as Content-based Match

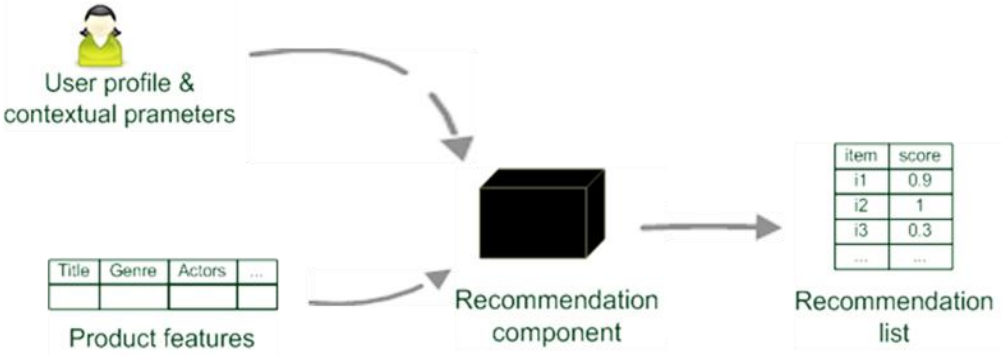
- Basic Idea
  - Recommend items similar to what the user liked before
  - Use item attributes or descriptions as the basis of recommendation
  - Build a user profile from past preferences
  - Match user interests with item content

"show me more of the same what I've liked"



- Needs:
  - Item information
    - For movie items: genre, actors, director, plot keywords, and user review etc.
    - For article items: title, author, category, tags etc.
  - User profile
    - Item consumption history

- The task:
  - Extract user preference from the consumption history
  - Identify items that are "similar" to the user's preference



Build Content-based Recommender

- Similarity-based approach
  - Compute the similarity of an unseen item with the user profile based on the keyword overlap with one metric, such as cosine similarity, or use and combine multiple metrics
- Learning-based approach
  - Train a predictive model using features and labels
    - Training data: <features, label>
    - Prediction: <features, ?>
- Both rely on good item representation

Content Representation and Similarities

- Item representation

Title	Genre	Author	Type	Price	Keywords
The Night of the Gun	Memoir	David Carr	Paperback	29.90	Press and journalism, drug addiction, personal memoirs, New York
The Lace Reader	Fiction, Mystery	Brunonia Barry	Hardcover	49.90	American contemporary fiction, detective, historical
Into the Fire	Romance, Suspense	Suzanne Brockmann	Hardcover	45.90	American fiction, murder, neo-Nazism

★ A simple approach (similarity-based):

- Compute the similarity of an unseen book item with the one in the user profile based solely on the keyword overlap using cosine similarity

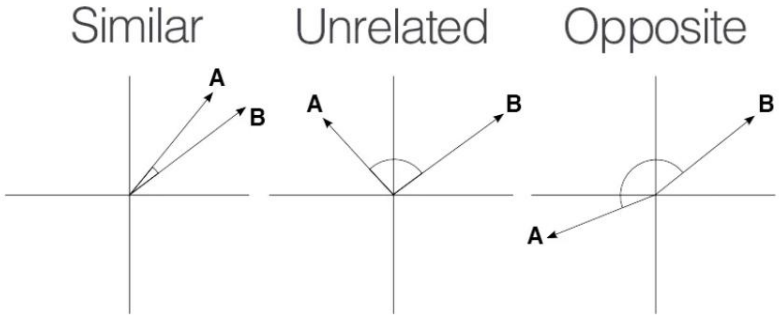
- User profile

Title	Genre	Author	Type	Price	Keywords
…	Fiction	Brunonia, Barry, Ken Follett	Paperback	25.65	Detective, murder, New York

- Definition

Cosine Similarity, a metric used to measure the angle between two non-zero vectors in a multi-dimensional space

$$\cos(\theta) = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$



The cosine similarity value ranges from -1 (completely dissimilar) to 1 (completely similar), with 0 indicating for no similarity

Cosine Similarity

- Computing Example

  Assume a vocabulary of 6 keywords, each book item can be described as a vector based on the occurrence of each keywords:

  [press, addiction, memoirs, New York, fiction, detective]

  Book A = [1, 1, 1, 1, 0, 0] User U = [0, 0, 0, 1, 0, 1]

  Book B = [0, 0, 0, 0, 1, 1]

  Book C = [0, 0, 0, 0, 1, 0]

  Using Cosine Similarity, we find that cos(U, B) > cos(U, A) > cos(U, C)

Keywords
Press and journalism, drug addiction, personal memoirs, New York
American contemporary fiction, detective, historical
American fiction, murder, neo-Nazism

Keywords
Detective, murder, New York

$$\cos(\theta) = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

Recommend

- Top-K ranking
  - Due to limitations in space and attention, we would like to prioritize the recommended items based on the obtained rank list.

Given a ranked list B > A > C for user U, we can make recommendations to the user as follows:

If K = 1, then we recommend [B]

If K = 2, then we recommend [B, A]

…

Example: Game Apps Recommendation

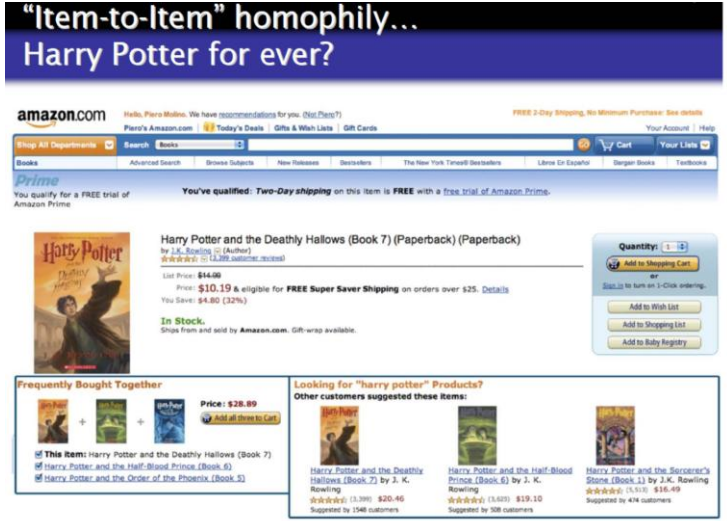
- Game App description:
  - <Developer, Category, Price, Language, Seller, Description …>
- User profile: based on what game apps (s)he downloaded or played before
  - <Developer, Category, Price, Language, Seller, Description …>

Advantages of Content-based RS (CBRS)

- User Independence
  - CBRS utilizes solely ratings provided by the active users to build her own profile
  - No need for data on other users
- Transparency
  - CBRS can provide explanations for recommended items by listing content-features that caused an item to be recommended
- New Items (Item not yet consumed or rated by any user)
  - CBRS are capable of recommending new items as long as their attributes are given
  - No cold-start problem

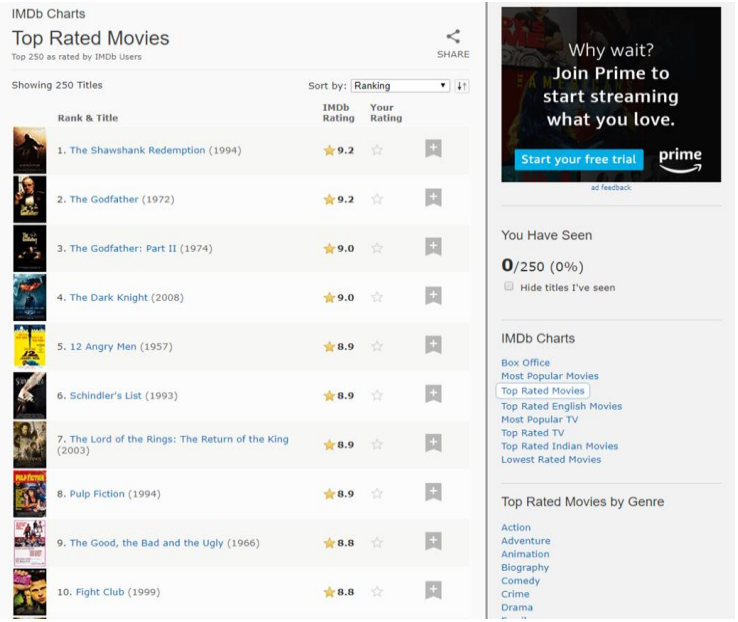
Limitations of CBRS

- Content-based filtering tends to recommend similar items to the user's past, which can lead to limited diversification in recommendations.
- Heavily depends on accurate item feature extraction, and if the features are not well-defined or incomplete, it may result in poor recommendations.
- Does not consider the opinions of other users, which can limit the discovery of serendipitous (interesting) recommendations.



Recommendation as Rating-based Ranking

- Transition: Why move beyond content?
  - What if item descriptions are incomplete or weak?
  - What if users would like items that do not look obviously similar?
  - What if crowd behavior reveals patterns that content alone cannot capture?
- Recommendation is usually a ranking task, where
  - Users provide feedback ratings to signal relevance
  - System predicts item ratings to a user
  - Items are sorted by highest to lowest ratings
  - In practice, only a few items are shown



Collaborative Filtering: Basic Idea

- Use the behavior of many users to make recommendations
- Learn from explicit or implicit feedback behaviors (e.g., ratings, likes, clicks, purchases, or interactions)
- The Core Design Intuitioin:
  - Similar users may prefer similar items
    - Often described as using the “wisdom of the crowd”

Approach the CF Recommender

- Assume:
  - Explicit feedback (e.g., numeric ratings)
  - A matrix of user–item ratings

	i1	i2	i3	i4	i5
u1	5	3		?	
u2		4		2	
u3	3	5	5		
u4				4	2
u5		4		3	

- Output:
  - A predicted score, indicating to what degree the current user will like or dislike a certain item
  - A top-*N* list of recommended items for each user

- Rows represent users
- Columns represent items
- Cells contain ratings
- Empty cells to predict
- Sparsity is a challenge

	i1	i2	i3	i4	i5
u1	5	3		?	
u2		4		2	
u3	3	5	5		
u4				4	2
u5		4		3	

CF: User-based Model

- Approach
  - Start with an "active user" (namely, Alice) and an item i not yet seen by Alice
    - find a set of users (known as neighbors) who liked the items rated by Alice and have rated item i
  - Apply an aggregated score of their ratings (e.g., the average) to predict how Alice will like item i
  - Repeat for all items that Alice has not seen, rank them, and recommend the top-rated one(s)
- Rationale
  - If users had similar tastes in the past they will have similar tastes in the future
  - User preferences remain stable and consistent over time

- Example

A feedback matrix of ratings by Alice, along with other users is given as shown below:

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

We need to determine how would Alice like Item5, which Alice has not yet seen (hence it remains unrated)

- Initial questions
  - How do we measure similarity among users?
  - How many should we consider in the neighborhood?
  - How do we generate a prediction from the neighbors' ratings?

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

Measuring User Similarity

- A popular similarity measure in user-based CF: Pearson correlation

  a, b : two users

  $r_{a,p}$: rating of user a for item p

  P : a set of items that is co-rated by user a and b

$$sim(a,b) = \frac{\sum_{p \in P}(r_{a,p} - \underline{\bar{r}_a})(r_{b,p} - \underline{\bar{r}_b})}{\sqrt{\sum_{p \in P}\left(r_{a,p} - \bar{r}_a\right)^2}\sqrt{\sum_{p \in P}\left(r_{b,p} - \bar{r}_b\right)^2}} \in [-1,1]$$

Note: Average ratings are computed from all available ratings for each user

Making Predictions

- A common prediction function:

$$pred(a,p) = \overline{r_a} + \frac{\sum_{b \in N} sim(a,b) * (r_{b,p} - \overline{r_b})}{\sum_{b \in N} sim(a,b)}$$

- Calculate how much the ratings of neighbors N for the unseen item p are higher or lower than the active user's average $\bar{r}_a$
- Combine the rating differences – use the similarity with a as a weight
- Add/subtract the neighbors' bias from the active user's average and obtain the predicted rating

- For example, we need to compute for $sim(Alice, User2)$
  - Step 1: co-rated set $P = \{item1, item2, item3, item4\}$
  - Step 2: averages $\bar{r}_{Alice} = \frac{5+3+4+4}{4} = 4$ and $\bar{r}_{User2} = \frac{4+3+4+3+5}{5} = 3.8$
  - Step 3: $sim(Alice, User2) = \frac{(5-4)(4-3.8)+(3-4)(3-3.8)+(4-4)(4-3.8)+(4-4)(3-3.8)}{\sqrt{(5-4)^2+(3-4)^2+(4-4)^2+(4-4)^2}\sqrt{(4-3.8)^2+(3-3.8)^2+(4-3.8)^2+(3-3.8)^2}} = 0.61$

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

$$sim(a,b) = \frac{\sum_{p \in P}(r_{a,p} - \bar{r}_a)(r_{b,p} - \bar{r}_b)}{\sqrt{\sum_{p \in P}(r_{a,p} - \bar{r}_a)^2}\sqrt{\sum_{p \in P}(r_{b,p} - \bar{r}_b)^2}}$$

- Step 4: Repeating the computing for other users, so we have the similarities between Alice and others are obtained as follows
- Step 5: Assume a threshold of neighborhood size = 2, we have $N = \{User1, User2\}$
- Step 6: Apply the prediction function:

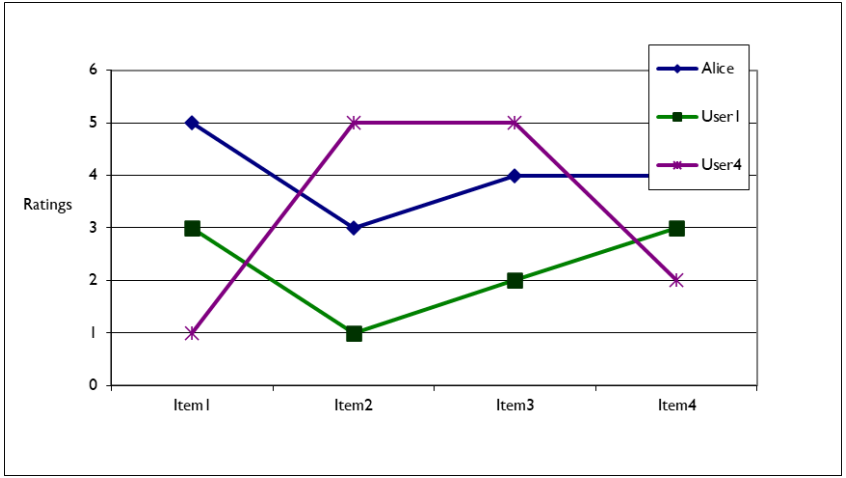
$$Pred(Alice, Item5) = 4 + \frac{0.84 * (3 - 2.4) + 0.61 * (5 - 3.8)}{0.84 + 0.61} = 4 + 0.85 = 4.85$$

	Item1	Item2	Item3	Item4	Item5	
Alice	5	3	4	4	?	
User1	3	1	2	3	3	0.84
User2	4	3	4	3	5	0.61
User3	3	3	1	5	4	0
User4	1	5	5	2	1	- 0.77

$$pred(a,p) = \overline{r_a} + \frac{\sum_{b \in N} sim(a,b) * (r_{b,p} - \overline{r_b})}{\sum_{b \in N} sim(a,b)}$$

Pearson Correlation is Useful

- Focuses on relative preferences rather than raw scores
- Especially useful when users apply different rating scales
- Considers rating biases across users



CF: Item-based Model

- Basic idea:
  - Use the similarity between items (instead of users) to make predictions
- Example:
  - Look for items that are similar to Item5
  - Take Alice's ratings for these items to predict the rating for Item5

	Item1	Item2	Item3	Item4	Item5
Alice	5	3	4	4	?
User1	3	1	2	3	3
User2	4	3	4	3	5
User3	3	3	1	5	4
User4	1	5	5	2	1

The cosine similarity measure

- Cosine similarity applies to the column vectors for item i and j

$$sim(i,j) = \frac{\sum_{u \in U} r_{u,i} * r_{u,j}}{\sqrt{\sum_{u \in U} r_{u,i}^2} \sqrt{\sum_{u \in U} r_{u,j}^2}}$$

In which, U denotes the set of users who have rated both items i and j

Making Predictions

- A common prediction function:

$$pred(u,j) = \frac{\sum_{i \in ratedItem(u)} sim(i,j) * r_{u,i}}{\underline{\sum_{i \in ratedItem(u)}} sim(i,j)}$$

- Size of neighborhood (rated items by user u) is also limited to a pre-specified number
- Not all item neighbors have to be considered
- An analysis of the MovieLens dataset indicates that
  - *"in most real-world situations, a neighborhood of 20 to 50 neighbors seems reasonable"*

Why Item-based filtering became popular

- Item relationships may be more stable than that of users
- Similarities can often be pre-computed offline
  - For example of Amazon Inc., items' similarities are computed during the off-peak hours; given a smaller number of ratings, the array of pairwise similarities between items quickly stabilizes
- Runtime recommendation then uses only a small neighborhood of items to predict

CBRS vs. Collaborative Filtering

- Content-based RS
  - uses item attributes
  - works well for new items
  - easier to understand and explain
- Collaborative Filtering
  - uses crowd behavior
  - can capture deeper taste patterns
  - does not require rich item descriptions
- Choice depends on data availability and business needs

CF: Latent Factor Model

- Motivations: Why do we go beyond neighborhood-based methods?
  - User- and Item-based Filtering depends on direct similarity neighborhood
  - Real preferences may be driven by hidden structures rather than neighbors
  - Users and items may be connected through unobserved taste dimensions

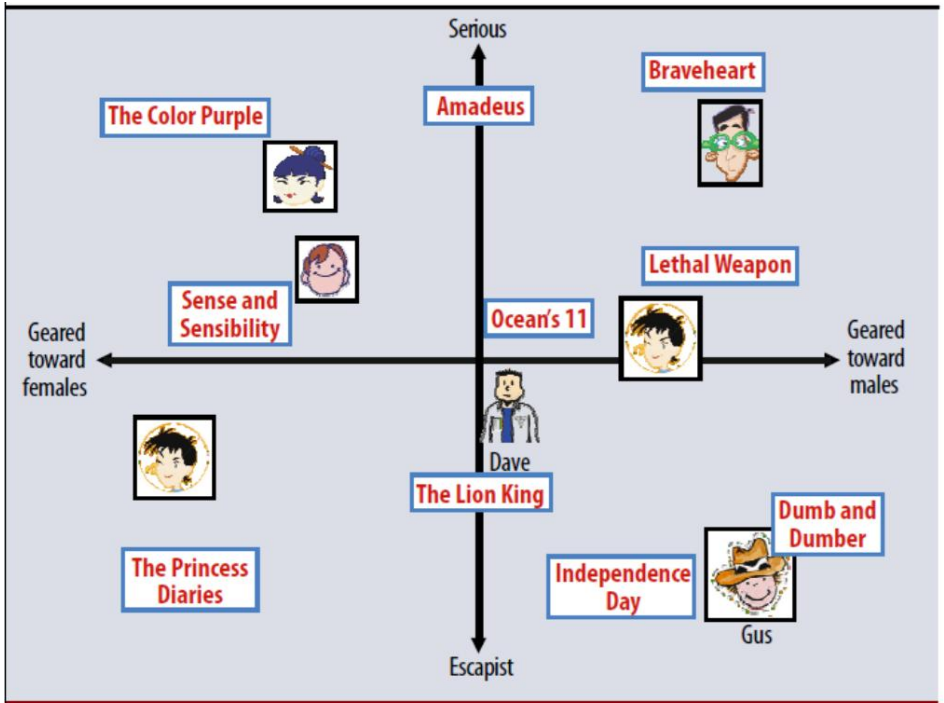
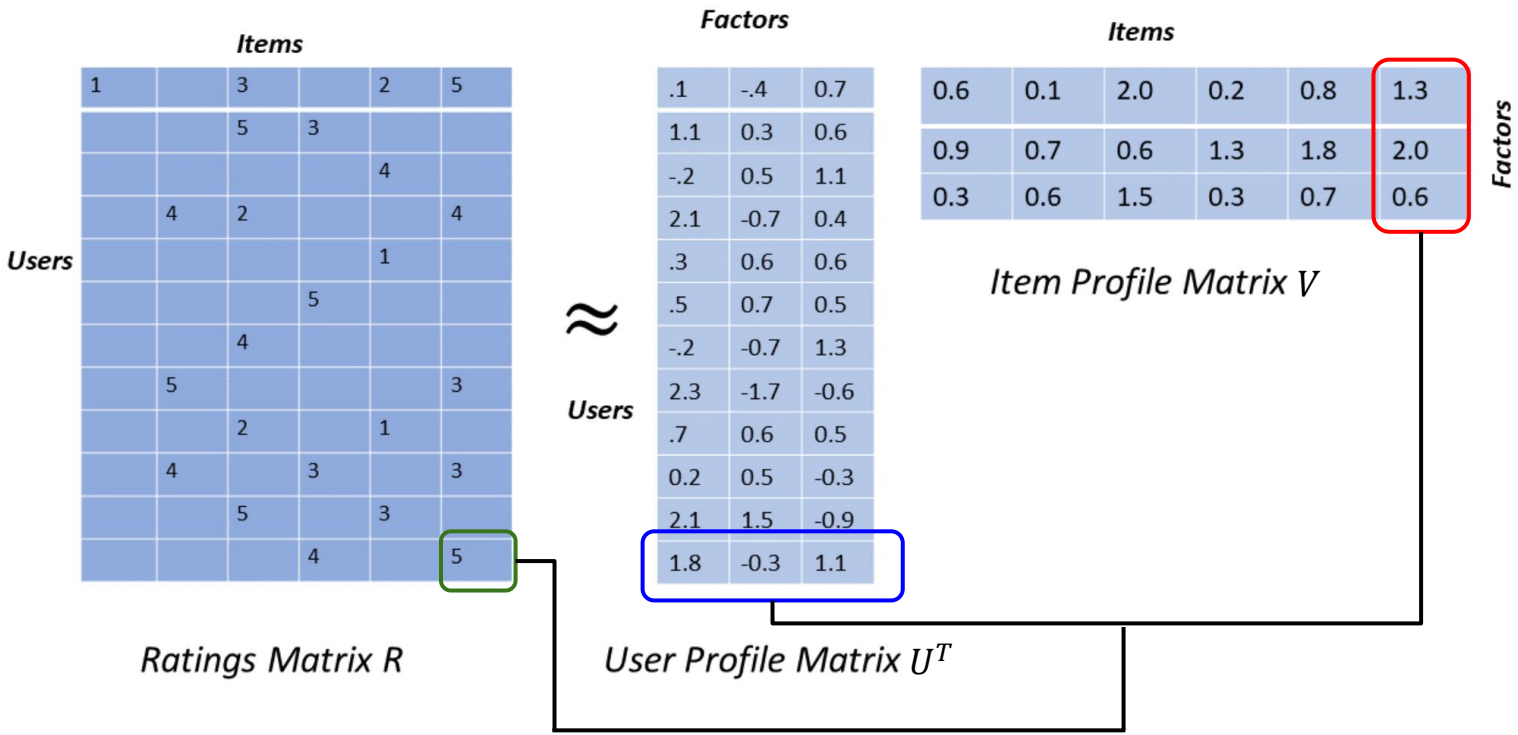


Figure 2. A simplified illustration of the latent factor approach, which characterizes both users and movies using two axes—male versus female and serious versus escapist.

Latent Factor Model: Matrix Factorization



Making Predictions

The predicted rating for item *j* by user *i* is:

$$R_{ij} = U_i^T \cdot V_j$$

	Item1	Item2	Item3	…	Item M
User 1	2	?	4		1
User 2	?	8	6		?
User 3	2	?	?		7
…					
User N	1	5	4		2

Evaluating Recommender Systems

- A myriad of techniques has been proposed, but
  - Which one is the best in a given application domain?
  - What are the success factors of different techniques?
  - Comparative analysis based on an optimality criterion?

- Further questions are:
  - Is a RS efficient with respect to a specific criteria like accuracy, user satisfaction, response time, serendipity, online conversion, ramp-up efforts, ….
  - Do customers like/buy recommended items?
  - Do customers buy items they otherwise would have not?
  - Are they satisfied with a recommendation after purchase?

Evaluation Settings

- Offline evaluation by using historical data

- Online evaluation by control and treatment groups (A/B testing)

Offline Evaluation

- Historical Data
  - Training
  - Testing

item →

user ↓

1		3		2	2
	1	2	4	4	
1		1	1		1
	3		3	4	2

Origianl Dataset

1		X		X	2
	1	2	X	X	
1		1	1		1
	3		3	X	X

Training Dataset

		3		2	
			4	4	
				4	2

Testing Dataset

Performance Metrics: Precision and Recall

- Recommendation is viewed as information retrieval task:
  - Retrieve (recommend) all items which are predicted to be “good”.
- Precision: a measure of exactness, determines the fraction of relevant items retrieved out of all items retrieved
  - E.g. the proportion of recommended movies that are actually good

$$Precision = \frac{tp}{tp + fp} = \frac{|good\ movies\ recommended|}{|\text{all recommendations}|}$$

- Recall: a measure of completeness, determines the fraction of relevant items retrieved out of all relevant items
  - E.g. the proportion of all good movies recommended

$$Recall = \frac{tp}{tp + fn} = \frac{|good\ movies\ recommended|}{|all\ good\ movies|}$$

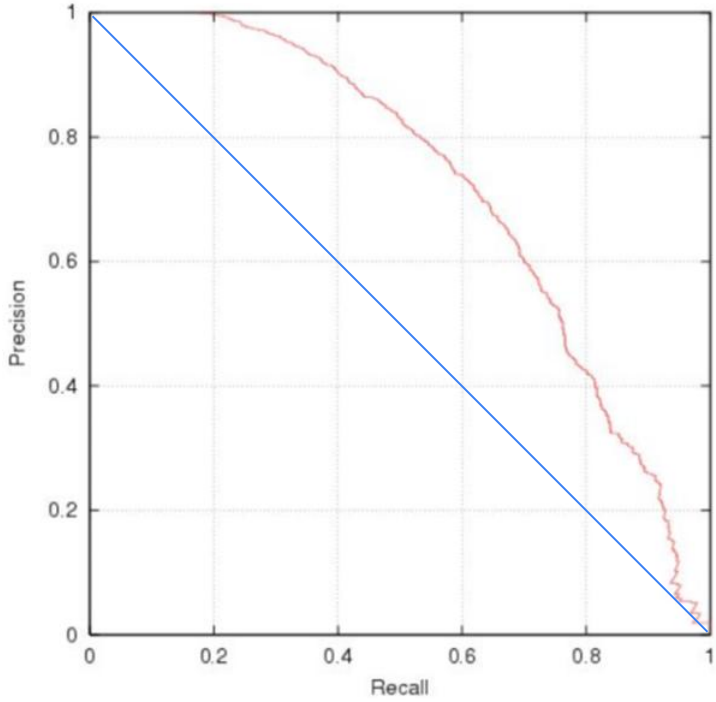
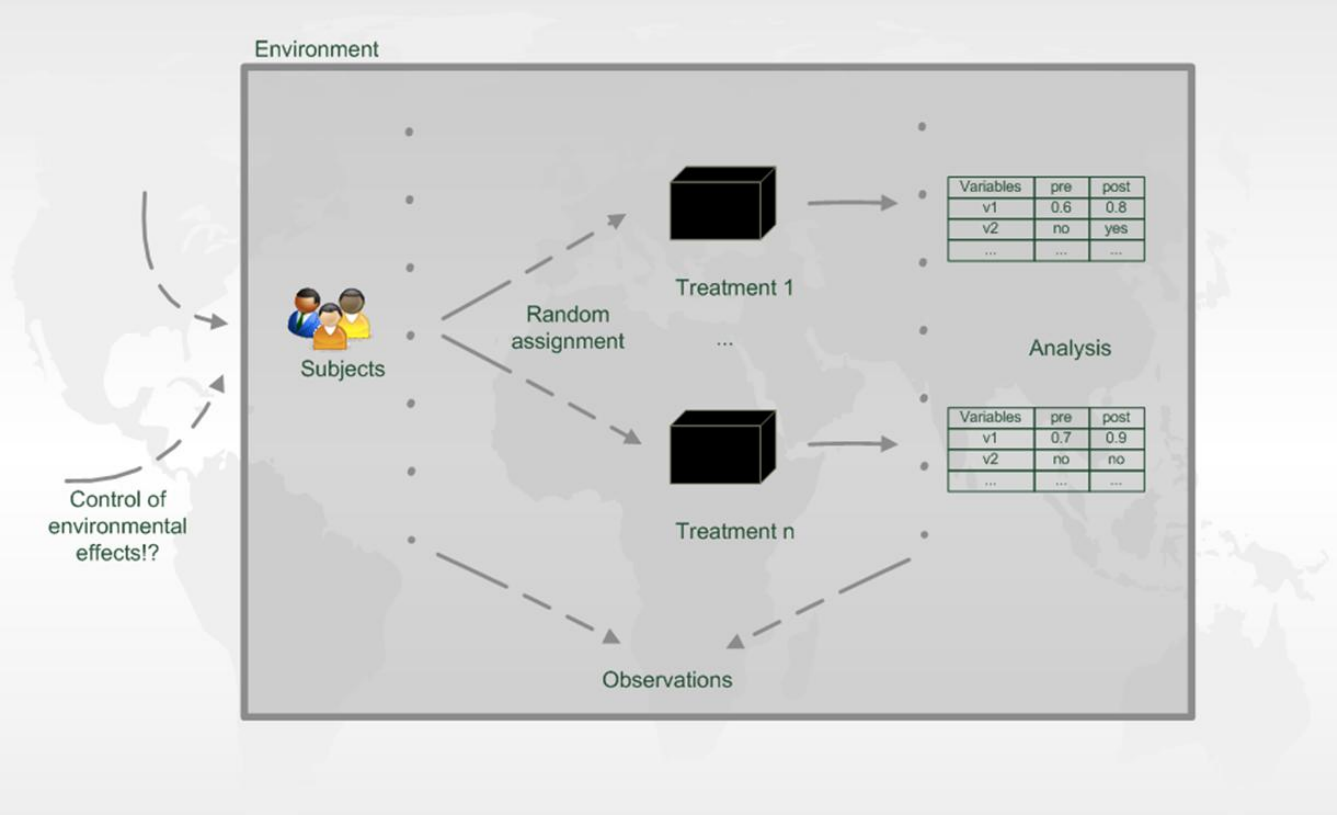
		Items Not Recommended	Items Recommended
		0	1
Irrelevant	0	TN	FP
Relevant	1	FN	TP

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

Precision vs. Recall

- Given item scores for users, tuning a recommender system to increase precision, will decrease recall as a result (or vice versa)
  - To increase precision, we should only make recommendation when being very confident. Thus, it is better off to raise the bar of recommendation by using a larger cutoff threshold of the predicted score. ↑
  - To increase recall, it is better off to make as much recommendations as possible. Such that there is more chance to hit the “right answer”. Thus, it is reasonable to use a lower cutoff threshold of the predicted score. ↓


Precision
Recall
0
0.2
0.4
0.6
0.8
1

Online Evaluation